ICT374 Lab 8:
Breaking Up Command Line into a Sequence of Commands (cont.); Network programming with TCP and UDP sockets.

Learning Objectives:

1. Learn to program in sockets using UNIX domain protocol and TCP and UDP protocols.

Required Reading:

- Lecture Notes for Topic 8
- Stevens and Rago: Ch 13 & Ch 16

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself, is under a directory called `lab08`

1. **Separating a command line into a sequence of commands - standard input/output redirections**

   This exercise is relevant to Project 1 - A Simple Unix Shell.

   Continued from C Programming Exercise 2 in Lab 7,

   Add the following three additional fields to the `CommandStructure` structure:

```
struct CommandStructure {
    int first;          // index of the first token of
                        // the command in the array of tokens
    int last;           // index of the last token of
                        // the command in the array of tokens
    char* separator;    // the command separator that follows
                        // the command. It should be
                        // one of the following
                        //  "|"   - pipe  to the next command
                        //  "&"   - shell does not wait
                        // for this command
                        //  ";"   - shell wait for this command
    // new fields
    char **argv;        // an array of tokens that forms a command
    char *stdin_file;   // if not NULL, points to the file name
                        // for stdin redirection
    char *stdout_file;  // if not NULL, points to the file name
                        // for stdout redirection
}
```

   Add the following function to module `command`:

```
void searchRedirection(char *token[], Command *cp);
```

This function takes an array of tokens, `token`, and a pointer to a single command structure, `cp`. It scans the array from `token[cp->first]` to `token[cp->last]`, looking for the standard input redirection symbol "<" and the standard output redirection symbol ">". Once found, the token following the redirection symbol is treated as the redirection file name and is assigned to either `cp->stdin_file` if it is the standard input redirection, or to `stdout_file` if it is the standard output redirection.

Note each command structure `c` should be initialized to the following values before calling the function `separateCommands`:

```
c.firs  = 0;
c.last  = 0;
c.separator = NULL;
c.argv  = NULL;
c.stdin_file = NULL;
c.stdout_file = NULL;
```

Therefore, if no redirection symbol is found, the two fields `cp->stdin_file` and `cp->stdout_file` should have value NULL.

Call the function `searchRedirection` inside the following function

```
int separateCommands(char *tokens[], Command command[]);
```

just before the function `separateCommands` returns to handle the standard input/output redirection for *each* command structure before `separateCommands` returns.

2. **Separating a command line into a sequence of commands - building command line argument array `argv`**

   This exercise is relevant to Project 1 - A Simple Unix Shell.

   Add the following function to module `command`:

   ```
   void buildCommandArgumentArray(char *token[], Command *cp);
   ```

   This function dynamically allocates an array of *n* char pointers for `cp->argv`:

   ```
   argv = realloc(argv, sizeof(char *) * n);
               // should check realloc's return value
   ```

   where *n* is the number of command line arguments for that command:

   ```
   n = (cp->last - cp->first + 1)
                       // the number of tokens in the command
       - ( 2 if cp->stdin_file is not NULL)
                       // remove 2 tokens for stdin redirection
       - ( 2 if cp->stdout_file is not NULL)
                       // remove 2 tokens for stdout redirection
       + 1;            // the last element in argv must be a NULL
   ```

   The function then assigns the address of each token to the corresponding array element of the array `cp->argv`:

   ```
   int i;
   int k = 0;
   for (i=cp->first; i<= cp->last; ++i ) {
   ```

```
        if (strcmp(token[i], ">") == 0
              || strcmp(token[i], "<") == 0)
            ++i; // skip off the std in/out redirection
        else {
            cp->argv[k] = token[i];
            ++k;
        }
    }
    cp->argv[k] = NULL;
```

Modify the function:

```
int separateCommands(char *tokens[], Command command[]);
```

so that immediately after calling the function:

```
void searchRedirection(char *token[], Command *cp);
```

for a command, you call the function `buildCommandArgumentArray` for the same command structure for each command structure before the function returns. The following lines should be added to the end of the function `separateCommands` (just before the return statement):

```
for (i=0; i<nCommands; i++) {
     searchRedirection(token, &(command[i]));
     buildCommandArgumentArray(token, &(command[i]));
}
```

Once the function `separateCommands` finally returns, you will have an array of command structures and each command structure, `command[i]`, contains the information that is almost ready for use (except wildcards handling) to execute the command with system call `execvp`:

```
execvp(command[i].argv[0], command[i].argv);
```

The improved module is available here: command.h and command.c.

A Special Note:

1. Each of the following exercises consists of a client and a server. Except the first exercise, all the other servers are daemons. They are designed to keep running even after you have logged out from the machine. In order not to burden the lab machines with excessive numbers of processes, some of which may even be buggy, you should remove those daemon processes before you log out each time.

   You should record the server pid number of each daemon process you have started. At the end of your exercise you should kill them using the command:

   ```
   kill -9 pid_of_the_daemon
   ```

   This is especially important if you run your daemons on the Eris server which is used by many units across several countries. You ***must*** kill your daemons before logging out of the Eris server.

   To record the pid of your daemon process, you should add the following line in your server programs:

```
printf("Daemon pid = %d\n", pid);
```

in the function "daemon_init":

```
void daemon_init(void)
{
     pid_t   pid;

     if ( (pid = fork()) < 0) {
          perror("fork");
          exit(1);
     } else if (pid != 0) {
          printf("Daemon pid = %d\n", pid); // pid of your daemon
          exit(0);        /* parent goes bye-bye */
     }

     /* child continues */
     setsid();               /* become session leader */

     chdir("/");             /* change working directory */

     umask(0);               /* clear our file mode creation mask */
}
```

2. If you run the server program on eris, you should use the port number assigned to you. To see the TCP port and UDP port allocated to you on eris server, please visit Unit Information page on this LMS.

Unix Programming Exercises

The examples in the following exercises refer to the examples in Topic 8.

1. **Example 1 - Unix domain protocol**

   Copy Example 1 (`ser1.c` and `cli1.c`) to your home directory. The source code is available from Topic 8 Examples directory. Read the example carefully to fully understand it. Then compile `ser1.c` and `cli.c`. If the compilation is successful, run the server first and then the client.

2. **Example 2 - Unix domain protocol again**

   Do the same for Example 2 (`ser2.c` and `cli2.c`). I recommend that you change the server socket address to a filename under your home directory to avoid clashes with other users' processes (in case there is more than one user on the machine you are using).

3. **Example 3 - UDP protocol**

   Try Example 3. Again choose a right socket name to avoid clashes with other users.

4. **Example 4 - TCP protocol**

   Try Example 4. However you must modify the server IP number to that of the machine you are using. In this example, you will need to manually convert the dotted decimal to a decimal. If the dotted decimal is A.B.C.D, the decimal would be

   ```
   decimal = A*256*256*256 + B*256*256 + C*256 + D
   ```

You may also need to change the port number to something that does not clash with others (for example use the first 5 digits in your student number). The port number must be over 1024.

You can run the client and the server on the same machine or on two different machines. One way to run them on different machine is to use eris (eris.ad.murdoch.edu.au) for running your server and your virtual machine (or home Linux machine) for running the client. Alternatively, you can run the server in one virtual machine and run the client in another virtual machine (log in two separate Windows machines in the lab).

5. **Example 5 - use `gethostbyname` and `INADDR_ANY`**

   Try Example 5. I recommend that you send some harmless signals (such as SIGUSR1) to the server to see if it can be interrupted.

   Try this example on the same machine. In the client use "localhost" to refer to the server machine.

   Then try the same example on two machines. For example one on eris and the other on a virtual machine, or on two virtual machines. Make sure you find out the server host IP first (type command `/sbin/ifconfig`).

6. **Example 6 - stream read and write**

   Try Example 6.

   Run the server and the client on two different machines.

7. **Example 8 - UDP protocol**

   Try Example 8 by running the client and the server on different machines.

The Weekly Lab

Submit the following exercises:

1. Complete C Programming Exercise 2.
2. Complete Unix Programming Exercise 4.
3. Complete Unix Programming Exercise 6
4. Complete Unix Programming Exercise 7

This weekly lab must be submicodeed to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submicodeing Weekly Labs when preparing and submicodeing a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 25 September 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming

Last Modified: Thursday, 25-Sep-2025 16:17:02 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only